Autores: Hugo Santos / Henrique Antunes / Pedro Oleirinha

1 Keyboard Reader

O módulo *Keyboard Reader* é a unidade responsável pela interação direta com o utilizador, tendo como função principal a descodificação do teclado matricial de 16 teclas. O seu objetivo é detetar a pressão de qualquer tecla e disponibilizar o respetivo código binário de 4 bits (K 3:0) ao módulo de controlo do sistema

Embora a arquitetura completa do módulo preveja a inclusão de um **Ring Buffer** (para armazenamento de até 18 códigos) e de um **Key Transmitter** (para envio série dos dados ao software), nesta fase inicial de desenvolvimento, o Keyboard Reader é constituído exclusivamente pelo bloco *Key Decode*.

Este bloco *Key Decode* integra duas sub-unidades fundamentais:

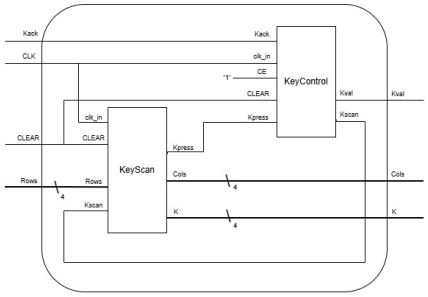
- **Key Scan**: Responsável pelo varrimento físico das linhas e colunas do teclado para identificar a tecla premida.
- **Key Control**: A unidade de controlo que gere o fluxo de dados através de um protocolo de handshake.

Nesta fase, a comunicação entre componentes é direta, ou seja, quando uma tecla é detetada, o sinal **Kval** é ativado e o código da tecla é colocado no barramento **K3:0**. O ciclo de leitura apenas é concluído após o sistema consumidor responder com o sinal de reconhecimento (**Kack**) e a tecla ser fisicamente libertada pelo utilizador, garantindo a integridade da leitura

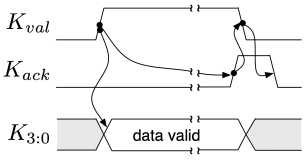
2 Key Decode

O bloco *Key Decode* implementa um descodificador de um teclado matricial 4x4 por hardware, sendo constituído por três sub-blocos: *i*) um teclado matricial de 4x4; *ii*) o bloco Key Scan, responsável pelo varrimento do teclado; e *iii*) o bloco Key Control, que realiza o controlo do varrimento e o controlo de fluxo, conforme o diagrama de blocos representado na Figura 1a.

O controlo de fluxo de saída do bloco *Key Decode* (que para uma fase futura será o módulo *Key Buffer*), define que o sinal K_{val} é ativado quando é detetada a pressão de uma tecla, sendo também disponibilizado o código dessa tecla no barramento $K_{3:0}$. Apenas é iniciado um novo ciclo de varrimento ao teclado quando o sinal K_{ack} for ativado e a tecla premida for libertada. O diagrama temporal do controlo de fluxo está representado na Figura 1b.



(a) Diagrama de blocos



(b) Diagrama temporal

Figura 1: Bloco *Key Decode*

2.1 KeyScan

O bloco *Key Scan* foi implementado de acordo com o diagrama de blocos representado na Figura 2.

A escolha da primeira opção para a implementação do bloco Key Scan, como está exposto na Figura 2, foi baseada numa análise de compromisso entre complexidade, tempo de desenvolvimento e boas práticas adquiridas na unidade do semestre passado em LSD.

Os principais motivos que justiçaram a nossa decisão foram. A reutilização de componentes, que conforme nos foi lecionado em LSD sempre que possível devemos reutilizar componentes já feitos. Onde no nosso caso esses componentes para além de já estarem feitos também já foram previamente validados e testados. Ao reutilizarmos estes componentes não só garantimos uma maior fiabilidade como também ganhamos tempo ao não precisar de testá-los.

Outro ponto foi a comparação entre análise de requisito vs o desempenho. Embora a nossa opção número 1 seja a menos eficiente em termos de ex-

ecução em comparação com as alternativas, chegámos á conclusão que para o contexto do nosso projeto Ticket Machine não seria relevante um maior desempenho de performance.

Para além que em termos de manutenção e futuros ajustes iria ser mais fácil de executar.

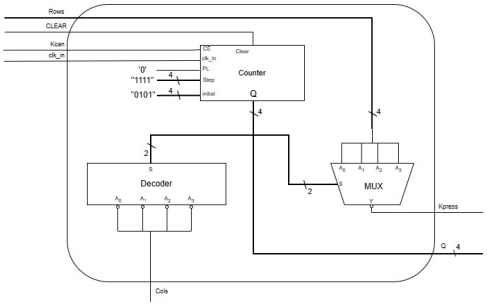


Figura 2: Diagrama de blocos do módulo *Key Scan*

2.2 KeyControl

A unidade de controlo (**Key Control**) foi implementada através de uma Máquina de Estados com três estados representada em *ASM-chart* na Figura 3, o que exigiu a utilização de dois bits (Q1,Q0) para a codificação do estado atual

A lógica de funcionamento detalha-se da seguinte forma:

- **Estado de Varrimento** (00): Nesta fase inicial, a saída incondicional **Kscan** permanece ativa, permitindo que o contador percorra sequencialmente a matriz de teclas. A máquina permanece neste estado enquanto a entrada **Kpress** (proveniente do MUX) for '0', indicando que nenhuma tecla foi premida.

- **Estado de Validação** (01): Assim que uma tecla é detetada (Kpress = '1'), o estado transita para o modo de leitura. Neste estado, a saída **Kval** é ativada para sinalizar ao sistema consumidor (software) que existe um código de tecla válido disponível no barramento **K 3:0**

- **Estado de Reconhecimento** (10): A transição para o último estado ocorre após a ativação do sinal de acknowledge (Kack = '1') pelo software, confirmando a receção dos dados. Neste estado, a máquina entra em espera (standby), aguardando que o sinal **Kack** retorne a '0' para garantir o cumprimento do protocolo de handshake

O ciclo de varrimento apenas é reiniciado após a conclusão deste protocolo e, conforme definido nos requisitos da primeira fase, após a libertação da tecla, garantindo que a mesma pressão não seja interpretada como múltiplas entradas. Este comportamento pode ser observado detalhadamente no diagrama temporal da Figura 1a

3 Conclusões

A implementação do módulo *Keyboard Reader*, nesta fase focada no bloco *Key Decode*, permitiu validar com sucesso a interface de hardware para a descodificação de um teclado matricial. A arquitetura adotada, baseada na separação funcional entre o varrimento (*Key Scan*) e o controlo (*Key Control*), demonstrou ser robusta e eficiente.

A utilização de uma Máquina de Estados (FSM) para garantir a integridade dos dados transmitidos ao software, revelou-se fundamental. O mecanismo de sincronismo entre os sinais Kval e Kack, assegura que cada pressão física resulte numa única leitura lógica, eliminando erros de duplicidade.

Embora a escolha pela reutilização de componentes de unidades curriculares anteriores (LSD) possa ter sacrificado marginalmente o desempenho teórico em termos de velocidade de varrimento, esta decisão provou ser uma estratégia de engenharia acertada para o projeto *Ticket Machine*.

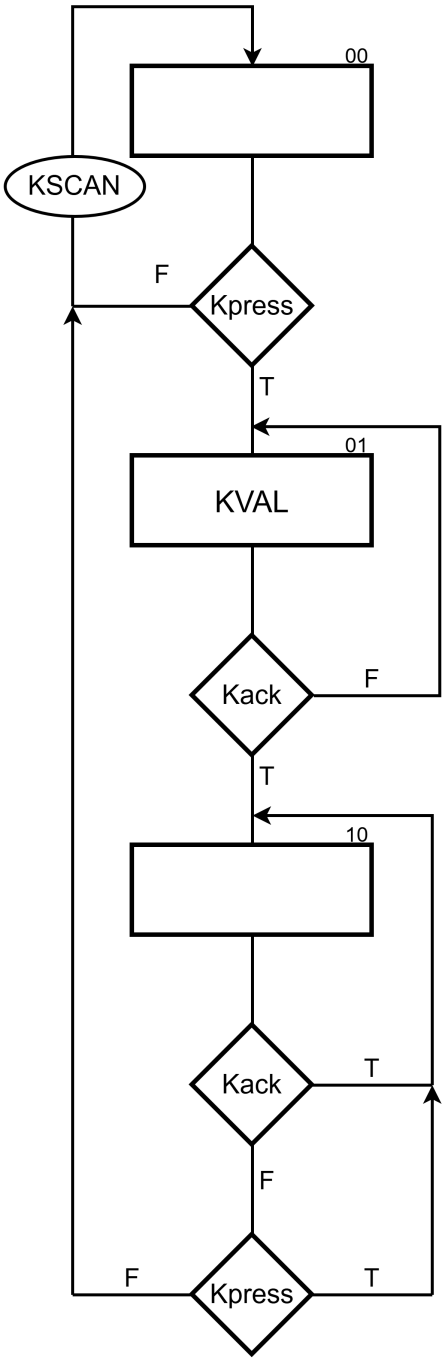


Figura 3: Máquina de estados do *Key Control*

A VHDL

A.1 Key Decode

```
library ieee;
use ieee.std_logic_1164.all;

ENTITY KeyDecode IS
        PORT(
                clk_in, Kack, CLEAR:         IN std_logic;
                rows:                                                   IN std_logic_vector(3 downto 0);
                cols:                                                   OUT std_logic_vector(3 downto 0);
                K:                                                           OUT std_logic_vector (3 downto 0);
                Kval:                                                          OUT std_logic
        );
END KeyDecode;

ARCHITECTURE Behaviour OF KeyDecode IS
        component KeyScan
                PORT(
                        clk_in, Kscan, CLEAR:IN std_logic;
                        rows:                                           IN std_logic_vector(3 downto 0);
                        cols:                                           OUT std_logic_vector(3 downto 0);
                        K:                                                   OUT std_logic_vector (3 downto 0);
                        Kpress:                                          OUT std_logic
                );
        end component;

        component KeyControl
                PORT(
                        clk_in, Kpress, Kack, CLEAR, CE: IN std_logic;
                        Kval, Kscan: OUT std_logic
                );
        end component;

        signal controlKpress, controlKscan : std_logic;

BEGIN

        scan: KeyScan port map(
                clk_in          => clk_in,
                Kscan          => controlKscan,
                rows                  => rows,
                cols                  => cols,
                CLEAR                 => CLEAR,
                K                          => K,
                Kpress          =>         controlKpress
        );

        control: KeyControl port map(
                clk_in         => clk_in,
                Kpress         => controlKpress,
                Kack                 => Kack,
                CLEAR                 => CLEAR,
                CE                         => '1',
                Kval                 => Kval,
                Kscan                 => controlKscan
        );


END Behaviour;
```

A.2 Key Scan

```
library ieee;
use ieee.std_logic_1164.all;

ENTITY KeyScan IS
        PORT(
                clk_in, Kscan, CLEAR: IN std_logic;
                rows:                           IN std_logic_vector(3 downto 0);
                cols:                           OUT std_logic_vector(3 downto 0);
                K:                                  OUT std_logic_vector (3 downto 0);
                Kpress:                          OUT std_logic
        );
END KeyScan;

ARCHITECTURE Behaviour OF KeyScan IS
        component Counter
                PORT(
                        clk_in, CE, CLEAR, PL: IN std_logic;
                        initial, step: IN std_logic_vector (3 downto 0);
                        Q: OUT std_logic_vector (3 downto 0);
                        Z: OUT std_logic
                );
        end component;

        component decoder
                PORT(
                        A: OUT std_logic_vector (3 downto 0);
                        S: IN std_logic_vector (1 downto 0)
                );
        end component;

        component MUX4_2L1
                PORT(
                        A: IN std_logic_vector(3 downto 0);
                        S: IN std_logic_vector(1 downto 0);
                        Y: OUT std_logic
                );
        end component;

        signal countValues: std_logic_vector (3 downto 0);
        signal zeros: std_logic;
        signal decodeOutputs: std_logic_vector (3 downto 0);


BEGIN

        cont: Counter port map(
                clk_in          => clk_in,
                CE                  => Kscan,
                CLEAR                 => CLEAR,
                PL                        => '0',
                initial         => "0000",
                step                 => "0001",
                Q                        => countValues,
                Z                        => zeros
        );

        muxPL: MUX4_2L1 port map(
                A                        => rows,
                S                        => countValues(1 downto 0),
                Y                        => Kpress

        );

        decode: decoder port map(
                A                         => decodeOutputs,
                S                         => countValues(3 downto 2)
        );

        cols <= decodeOutputs;
        K <= countValues;
```

A.3 Key Control

```
library ieee;
use ieee.std_logic_1164.all;

ENTITY KeyControl IS
        PORT(
                clk_in, Kpress, Kack, CLEAR, CE: IN std_logic;
                Kval, Kscan: OUT std_logic
        );
END KeyControl;

ARCHITECTURE Behaviour OF KeyControl IS

        type STATE_TYPE is (STATE_SCANNING, STATE_READING, STATE_ACKNOWLEDGE);

        signal currState, nextState: STATE_TYPE;
BEGIN

        currState <= STATE_SCANNING when CLEAR = '1' else nextState when rising_edge(clk_in);

generateNextState:
        process(Kpress, Kack, currState, nextState)
        begin
          nextState <= currState;

          case currState is
                        when STATE_SCANNING =>         if(Kpress = '1') then
                                                                                                                nextState <= STATE_READING;
                                                                                                        end if;
                        when STATE_READING =>         if(Kack = '1') then
                                                                                                                nextState <= STATE_ACKNOWLED
                                                                                                        end if;
                        when STATE_ACKNOWLEDGE =>         if(Kack = '0' AND Kpress = '0') then
                                                                                                                nextState <= STATE_SCANNING;
                                                                                                        end if;
          end case;
        end process;

        Kscan <= '1' when (currState = STATE_SCANNING AND Kpress = '0') else '0';
        Kval <= '1' when (currState = STATE_READING) else '0';


END Behaviour;
```

B *Kotlin*

B.1 HAL

Algoritmo 1: HAL - Hardware Abstract Layer

```
package org.example

import isel.leic.UsbPort

object HAL {
    var lastValue = 0

    // Inicia o objeto
    fun init() {
        lastValue = 0
        clrBits(0b11111111)
    }

    // Retorna 'true' se o bit definido pela mask est  com o valor l gico '1' no UsbPort
    fun isBit(mask: Int, value: Int? = null): Boolean {
        //O operador "and" faz uma compara  o AND bit a bit entre os dois valores
        return ((value ?: UsbPort.read()) and mask) == mask
    }

    // Retorna os valores dos bits representados por mask presentes no UsbPort
    fun readBits(mask: Int, value: Int? = null): Int {
        return (value ?: UsbPort.read()) and mask
    }

    // Escreve nos bits representados por mask os valores dos bits correspondentes em value
    fun writeBits(mask: Int, value: Int) {
        val allBits = mask and value
        val newValue = allBits or lastValue

        writeHAL(newValue)
    }

    // Coloca os bits representados por mask no valor l gico '1'
    fun setBits(mask: Int) {
        val newValue = lastValue or mask

        writeHAL(newValue)
    }

    // Coloca os bits representados por mask no valor l gico '0'
    fun clrBits(mask: Int) {
        val newValue = lastValue and mask.inv()
        writeHAL(newValue)
    }

    fun writeHAL(newValue: Int) {
        lastValue = newValue
        UsbPort.write(newValue)
    }
}
```

B.2 KBD

Algoritmo 2: KBD

```
package org.example

import isel.leic.utils.Time.getTimeInMillis
import isel.leic.utils.Time


// Ler teclas. Fun   es retornam '0'..'9', 'A'..'D', '#', '*' ou NONE.
object KBD {
    const val NONE = '_'


    val teclas = arrayOf(
        arrayOf('1', '2', '3', 'A'),
        arrayOf('4', '5', '6', 'B'),
        arrayOf('7', '8', '9', 'C'),
        arrayOf('*', '0', '#', 'D')
    )

    // Inicia a classe
    fun init() {
        SerialEmitter.init()
    }

    // Retorna de imediato a tecla premida ou NONE se n o h   tecla premida.
    fun getKey(): Char {
        val row = HAL.readBits(0b0011)
        val col = (HAL.readBits(0b1100) shr 2) //Signed Right Shift de dois bits
        println("Coluna: $col; Linha: $row")

        sendAckBit()

        return teclas[row][col]
    }

    fun sendAckBit() {
        HAL.setBits(mask = 0b10000000) //Define o ACK a 1
        println("ACK SENT a 1")
        Time.sleep(20)
        HAL.clrBits(mask = 0b10000000) //Limpa o ACK para finalizar o processo
        Time.sleep(20)
        println("ACK CLEAR.\n FINISHED")
    }

    fun isValOff(): Boolean {
        return !HAL.isBit(0b10000000)
    }

    // Retorna a tecla premida, caso ocorra antes do 'timeout' (em milissegundos),
    // ou NONE caso contr rio.
    fun waitKey(timeout: Long): Char {
        val time = getTimeInMillis() + timeout
        while (getTimeInMillis() < time) {

            if (!isValOff()) {
                val key = getKey()
                println("KEY: $key pressed")
                return key
            }
        }
        println("NO KEY PRESS")
        return NONE
    }
}
```